

Лекція 10. SonarQube

SonarQube – це потужна платформа для автоматичного аналізу якості коду, яка допомагає розробникам та командам забезпечувати високий рівень відповідності стандартам, мінімізувати технічний борг і підвищувати загальну надійність програмного забезпечення.

Мета створення SonarQube

Проблема: Ручний код рев'ю займає багато часу і є неефективним.

Рішення: Автоматичний інструмент для безперервного аналізу якості коду.

Мета SonarQube:

- Автоматизація процесу перевірки коду.
- Виявлення багів, вразливостей та коду з технічним боргом.
- Забезпечення стандартів кодування в командах.



SonarQube створений як універсальний інструмент для автоматизації процесу аналізу вихідного коду з акцентом на забезпечення високої якості програмного забезпечення. Його основною метою є оптимізація розробки через постійний моніторинг коду, ідентифікацію потенційних проблем та рекомендації щодо їх вирішення.

Основні цілі:

- *підвищення якості коду:*

SonarQube дозволяє виявляти широкий спектр проблем у коді, серед яких:

- *технічний борг:* недоліки коду, які знижують його читабельність, підтримуваність і ефективність. Це може включати погану структуру, застарілі практики та неефективний дизайн;
- *недоліки безпеки:* пошук уразливостей у коді, таких як SQL-ін'єкції, XSS (міжсайтовий скриптинг) та інші загрози;

- *дублікація коду*: виявлення повторюваних частин коду, що сприяє рефакторингу та спрощує підтримку;
- *помилки стилю*: аналізує відповідність коду стандартам стилю та рекомендаціям найкращих практик, що сприяє узгодженості коду в команді;
- *забезпечення відповідності стандартам*: SonarQube перевіряє код на відповідність міжнародним стандартам програмування, таким як CWE (Common Weakness Enumeration), OWASP (Top 10) та іншим. Це особливо корисно для проєктів, які потребують високого рівня безпеки чи сертифікації;
- *інтеграція в CI/CD процеси*: інструмент ідеально підходить для DevOps середовищ, оскільки його можна інтегрувати в пайплайни безперервної інтеграції та доставки:
 - під час кожного коміту чи пул-реквесту SonarQube виконує автоматичний аналіз, що дозволяє виявляти проблеми на ранніх етапах розробки, коли їх виправлення коштує значно менше;
 - розробники отримують зворотний зв'язок у режимі реального часу, що стимулює створення якісного коду ще на етапі написання;
- *зниження технічного боргу*: інструмент не тільки виявляє проблеми, але й надає розробникам чіткі рекомендації щодо їх усунення. Це дозволяє знизити витрати на подальше обслуговування та розвиток програмного забезпечення. Наприклад:
 - Заміна неефективних алгоритмів.
 - Покращення читабельності коду.
 - Усунення застарілих або нефункціональних частин.

Окрім основних цілей, SonarQube сприяє кращій командній співпраці, оскільки надає прозору картину стану проєкту, допомагаючи менеджерам приймати обґрунтовані рішення.

SonarQube підтримує понад 30 мов програмування, включаючи найпопулярніші, такі як Java, Python, C#, JavaScript, TypeScript, C++, Go, Kotlin

та інші. Це робить його універсальним рішенням для команд, які працюють із різними технологіями.

Інструмент легко інтегрується з популярними системами безперервної інтеграції та розгортання, такими як Jenkins, GitLab CI/CD, Azure DevOps, GitHub Actions тощо. Це дозволяє автоматично проводити аналіз коду на кожному етапі розробки.

SonarQube дозволяє налаштовувати профілі якості, правила та порогові значення для різних метрик відповідно до специфічних вимог проекту чи організації.

Завдяки інтеграції з DevSecOps практиками, SonarQube забезпечує високий рівень безпеки, виявляючи вразливості в коді ще до його розгортання.

Інструмент має зручний веб-інтерфейс, який дозволяє командам швидко переглядати результати аналізу, метрики якості, тенденції змін та рекомендації щодо покращення.

Ключові переваги використання SonarQube:

- *забезпечення відповідності стандартам якості коду*: завдяки чітко визначеним правилам та метрикам, SonarQube допомагає дотримуватися галузевих стандартів, таких як OWASP Top 10 для безпеки або ISO/IEC 25010 для якості програмного забезпечення;
- *рання діагностика проблем*: SonarQube дозволяє виявляти помилки, баги, вразливості, дублікації коду та проблеми з технічним боргом ще на ранніх етапах розробки, що значно знижує витрати на їх усунення;
- *підвищення продуктивності команди*: інструмент автоматизує процеси перевірки якості, звільняючи час розробників для роботи над новими функціональностями замість виправлення проблем у коді;
- *прозорість та контроль*: інструмент надає централізовану платформу для моніторингу стану коду всіх проектів, що дозволяє менеджерам приймати обґрунтовані рішення;

- *покращення безпеки*: завдяки автоматичному аналізу SonarQube виявляє потенційно небезпечний код, що може призвести до кіберзагроз, і допомагає усунути ці вразливості.

Особливості, що виділяють SonarQube серед конкурентів:

- Quality Gates дозволяють встановлювати пороги для ключових метрик, таких як покриття тестами, кількість багів або технічного боргу. Це допомагає командам автоматично приймати рішення щодо якості коду;
- SonarScanner – модуль аналізу, який виконує статичний аналіз коду, збирає метрики та передає результати на сервер SonarQube для подальшої обробки та візуалізації;
- SonarQube надає можливість зосередитися на перевірці змін у нових комітах або пул-реквестах, що є особливо корисним для великих проєктів;
- інструмент підтримує роботу з великими застарілими кодовими базами, дозволяючи поступово покращувати їхній стан через рефакторинг;
- розширюваність забезпечується завдяки плагінам, які дозволяють інтегрувати SonarQube з іншими інструментами чи сервісами, наприклад, SonarCloud для хмарних рішень.

SonarQube активно застосовують провідні компанії по всьому світу. ІТ-гіганти, такі як Google, Microsoft та Amazon, інтегрують його у свої процеси DevOps, щоб забезпечити стабільність та безпеку продуктів. У фінансовому секторі банки та страхові компанії використовують SonarQube для аналізу легасі-коду та дотримання нормативних вимог щодо безпеки.

У сфері мобільної розробки компанії використовують SonarQube для оптимізації продуктивності та стабільності своїх додатків. В індустрії розваг студії, які займаються розробкою ігор, застосовують інструмент для аналізу продуктивності, безпеки та якості коду своїх проєктів.

SonarQube – це більше, ніж просто інструмент для аналізу коду. Це невід’ємна частина процесів розробки, тестування та забезпечення якості, яка допомагає командам створювати надійне, безпечне та якісне програмне забезпечення. Його популярність зумовлена гнучкістю, потужною функціональністю та здатністю ефективно інтегруватися у сучасні DevOps-процеси.

Галузі застосування інструменту SonarQube

SonarQube є універсальним інструментом, що застосовується в багатьох галузях розробки програмного забезпечення та забезпечення його якості. Завдяки своїй функціональності, він допомагає командам розробників, інженерам DevOps, архітекторам програмного забезпечення та експертам із кібербезпеки забезпечувати надійність, безпеку та ефективність коду.

Галузі застосування SonarQube

Розробка програмного забезпечення:

- Автоматична перевірка комітів і pull request'ів.

Побудова безпечних систем:

- Виявлення вразливостей безпеки на ранніх етапах.

Контроль якості на проєктах:

- Використання у процесах DevOps для підтримки стабільності коду.

Навчання:

- Використання у навчальних проєктах для вивчення кращих практик програмування.



Розробка програмного забезпечення

SonarQube використовується для аналізу якості та відповідності коду в різних мовах програмування.

- *мови програмування*: підтримує понад 30 мов, зокрема Java, Python, C++, JavaScript, TypeScript, C#, PHP, Go, Ruby та багато інших;

- *модульність*: дозволяє командам розробляти великі та складні проекти, аналізуючи кожен модуль окремо;
- *етапи застосування*: використовується як на етапі розробки, так і для перевірки якості коду перед релізом.

Приклади:

- автоматичне тестування якості коду в Agile-процесах;
- використання у проєктах, що мають високі вимоги до продуктивності та масштабованості.

DevOps

Інтеграція SonarQube в CI/CD пайплайни робить його важливим інструментом для команд DevOps.

- *інтеграція з інструментами*: Jenkins, GitLab CI/CD, Azure DevOps, GitHub Actions, Bamboo, Bitbucket Pipelines тощо;
- *автоматизація процесів*: аналіз коду відбувається автоматично під час кожного коміту, пул-реквесту або створення білду;
- *запобігання проблемам*: якщо код не відповідає заданим Quality Gates, він не проходить далі у процесі розгортання.

Приклади застосування:

- у великих компаніях, які постійно оновлюють своє програмне забезпечення;
- у мікросервісних архітектурах для забезпечення стабільності кожного компонента.

Кібербезпека

SonarQube є важливим інструментом у сфері DevSecOps, оскільки допомагає виявляти та усувати вразливості в коді:

- аналіз безпеки для виявлення загроз, таких як SQL-ін'єкції, XSS, вразливі бібліотеки, порушення доступу до даних;
- інтеграція з OWASP Top 10 та CWE.

Приклади застосування:

- банківські системи та фінансові сервіси для забезпечення захисту даних клієнтів;
- розробка вебзастосунків із високими вимогами до безпеки.

Мобільна розробка

SonarQube забезпечує аналіз вихідного коду для мобільних платформ, таких як Android та iOS:

- підтримка мов Kotlin, Swift, Objective-C, Java;
- перевірка ефективності використання ресурсів мобільних пристроїв (пам'ять, процесор, батарея);
- виявлення проблем сумісності з різними версіями операційних систем.

Приклади застосування:

- розробка мобільних застосунків для e-commerce або фінансових сервісів;
- виявлення вузьких місць у продуктивності під час масштабного тестування.

Аудит існуючого коду

SonarQube використовується для аналізу великих кодових баз, що дозволяє оцінити їхній стан і знайти зони для рефакторингу:

- оцінка стану коду у застарілих проєктах (legacy systems);
- виявлення дублікацій, застарілих методів, помилок та вразливостей;
- зниження технічного боргу та спрощення підтримки.

Приклади застосування:

- реорганізація великих монолітних систем перед переходом до мікросервісної архітектури;
- підготовка до впровадження нових функцій у проєктах зі значною історією розробки.

Освітні та наукові дослідження

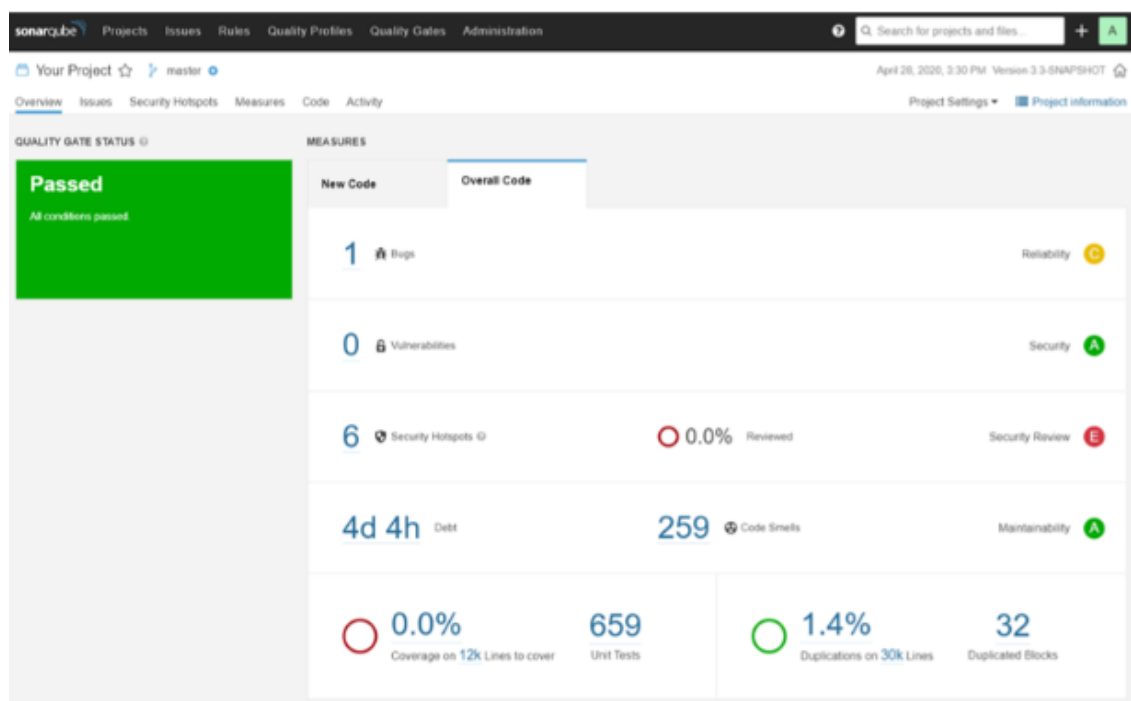
SonarQube також використовується в навчальних цілях та для досліджень у галузі якості програмного забезпечення.

Приклади:

- використання на курсах програмування для навчання принципам чистого коду;
- аналіз та покращення студентських проєктів;
- проведення наукових досліджень у галузі статичного аналізу коду.

SonarQube є потужним і універсальним інструментом, який знаходить своє застосування в широкому спектрі завдань і середовищ

На рисунку продемонстровано інтерфейс SonarQube для проєкту, в якому виконано аналіз коду.



Основні елементи та інформація, які можна побачити:

1. Quality Gate Status: результат аналізу позначений зеленим блоком із статусом «Passed», що означає, що всі умови якості успішно виконані.

2. Measures (Метрики)

Метрики поділені на дві частини: **New Code** (новий код) і **Overall Code** (загальний код).

- **New Code:**

- **1 Bug:** виявлена одна помилка;
- **0 Vulnerabilities:** вразливості відсутні;
- **6 Security Hotspots:** виявлено шість потенційно небезпечних ділянок коду, які потребують перевірки;
- **Overall Code:**
 - **4d 4h Debt:** обсяг технічного боргу складає 4 дні та 4 години;
 - **259 Code Smells:** виявлено 259 «запахів коду» (слабких місць або потенційних проблем);
 - **0.0% Coverage:** покриття тестами відсутнє (0% з 12 тисяч рядків коду);
 - **659 Unit Tests:** є 659 модульних тестів;
 - **1.4% Duplications:** 1.4% коду є дублікатами (на 30 тисяч рядків);
 - **32 Duplicated Blocks:** виявлено 32 дублікатних блоки коду.

3. Оцінки за категоріями

- **Reliability** (Надійність): оцінка «С» свідчить про середній рівень надійності коду;
- **Security** (Безпека): оцінка «А» означає високу безпеку;
- **Security Review** (Огляд безпеки): оцінка «Е» вказує на серйозні проблеми, які потребують уваги;
- **Maintainability** (Підтримуваність): оцінка «А» демонструє високу підтримуваність коду.

4. Додаткові елементи інтерфейсу

- у верхній частині екрана є вкладки для доступу до інших функцій: **Projects, Issues, Rules, Quality Profiles, Quality Gates, Administration;**
- також відображена дата та час аналізу: **April 28, 2020, 3:30 PM.**

Цей інтерфейс надає огляд стану проекту та дозволяє командам швидко ідентифікувати проблеми, фокусуючись на новому коді та ключових метриках.

Архітектура побудови SonarQube

SonarQube побудований на багаторівневій архітектурі, яка забезпечує гнучкість, масштабованість та інтеграцію з різними інструментами розробки. Кожен компонент виконує свою унікальну функцію, що дозволяє ефективно аналізувати код, обробляти дані та взаємодіяти з користувачами.

Архітектура SonarQube

Основні компоненти:

SonarQube Server:

- Зберігає базу даних проектів і результатів аналізу.

Database:

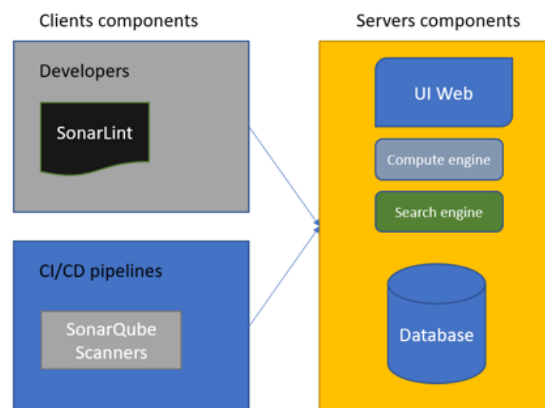
- Зберігання історії аналізів, помилок і метрик.

Web Interface:

- Панель для перегляду результатів аналізу та управління якістю коду.

SonarScanner:

- Інструмент для запуску аналізу коду з різних CI/CD систем.



1. Database Layer (Рівень бази даних)

Цей шар відповідає за збереження всіх даних, які створюються під час аналізу проекту.

- що зберігається:
 - метрики якості коду (кількість багів, дублікацій, рівень покриття тестами тощо);
 - логи та звіти з аналізу;
 - історія змін у проектах, яка дозволяє відстежувати прогрес команди;
- особливості:
 - підтримує реляційні бази даних, такі як PostgreSQL, MySQL або Oracle;
 - гарантує швидкий доступ до даних навіть у великих проектах з великими обсягами коду.

Приклад використання:

Зберігання історії якості проекту для аналізу прогресу в часі або оцінки впливу рефакторингу.

2. Server Layer (Серверний рівень)

Це центральний компонент SonarQube, який забезпечує обробку даних, взаємодію з іншими інструментами та управління проектами.

- *основні функції:*
 - отримання даних від сканера (SonarScanner) та їх обробка;
 - виконання правил аналізу, заданих Quality Gates;
 - надання API для інтеграції з іншими системами, такими як Jenkins або GitLab;
- *особливості:*
 - використовує потужний механізм обробки даних для аналізу великих проектів;
 - масштабується для підтримки великих команд і багатьох проектів.

Приклад використання:

Центральна точка для управління якістю коду у великих організаціях.

3. Scanner Layer (SonarScanner)

Цей шар є важливим компонентом, який відповідає за збір даних із вихідного коду.

- *призначення:*
 - виконує статичний аналіз вихідного коду;
 - збирає інформацію про структуру коду, залежності, коментарі, метрики, баги тощо;
- *можливості:*
 - підтримує різні мови програмування та білд-системи (Maven, Gradle, Ant тощо);
 - інтегрується з CI/CD пайплайнами для автоматизації аналізу;
 - може працювати з локальним кодом або кодом із репозиторію;

- *як працює:*

- збирає код із проєкту;
- виконує локальний аналіз;
- передає результати аналізу до серверного рівня.

Приклад використання:

Аналіз проєкту в реальному часі під час створення білду.

4. Web Interface (Веб-інтерфейс)

Інтуїтивний інтерфейс для перегляду результатів аналізу, управління проєктами та налаштування системи.

- *можливості:*

- *візуалізація метрик:* веб-інтерфейс відображає звіти про стан проєктів, динаміку змін, баги, уразливості та інші показники;
- *управління проєктами:* додавайте нові проєкти, налаштовуйте Quality Gates, переглядайте статус проєктів;
- *історія змін:* відстежуйте, як якість коду змінювалася з часом.

- *особливості:*

- забезпечує зручну навігацію по проєктах;
- підтримує багатокористувацький доступ із можливістю налаштування прав для різних ролей;
- дозволяє порівнювати поточний стан проєкту з попередніми версіями.

Приклад використання:

Перегляд звітів про аналіз коду для прийняття рішень щодо релізу.

5. Плагіни та розширення

Окрім основних компонентів, архітектура SonarQube підтримує систему плагінів, які розширюють його функціональність.

- *Приклади плагінів:*

- плагіни для специфічних мов програмування;

- інтеграція з ALM-системами (Application Lifecycle Management), такими як Jira або Azure DevOps;
- розширення для аналізу специфічних вимог безпеки.
- *можливості розширення:* користувачі можуть створювати власні плагіни для вирішення унікальних завдань.

SonarQube забезпечує високу масштабованість, дозволяючи ефективно працювати як для невеликих команд, так і для великих організацій із сотнями проєктів. Гнучкість системи досягається завдяки плагінам і API, що дозволяють адаптувати її до будь-яких потреб. Прозорість забезпечується через доступність усіх результатів аналізу у веб-інтерфейсі, що покращує взаємодію команди. Завдяки відкритим API та сумісності з багатьма інструментами DevOps, SonarQube ідеально інтегрується у сучасні процеси розробки.

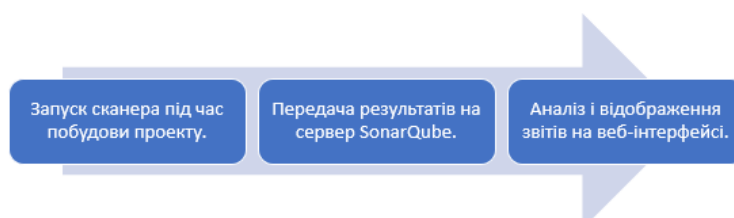
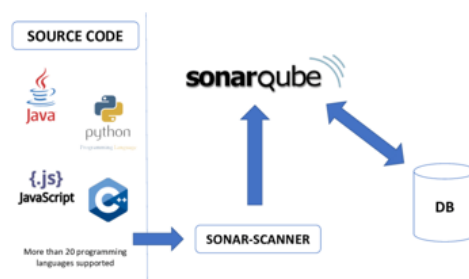
10.2. SonarScanner

SonarScanner — це ключовий компонент у екосистемі SonarQube, який використовується для збору, аналізу та передачі даних із вихідного коду до сервера. Він забезпечує виконання статичного аналізу проєктів незалежно від їхньої складності, обсягу та мови програмування.

Що таке SonarScanner?

SonarScanner — це основний інструмент для запуску аналізу коду у SonarQube.

- Може бути інтегрований у CI/CD системи (GitHub Actions, Jenkins, GitLab CI).
- Підтримує більше 25 мов програмування.



Основні цілі SonarScanner:

- збір метрик: отримання показників якості коду, таких як кількість багів, вразливостей, технічного боргу, дублікацій та інші;
- аналіз файлів: виконання статичного аналізу вихідного коду відповідно до налаштувань якості (Quality Profiles);
- передача даних: надсилання зібраних результатів на сервер SonarQube для подальшої обробки та візуалізації.

SonarScanner розроблений для роботи в різних середовищах розробки та підтримує широкий спектр технологій. Він є мультиплатформним інструментом, що працює на Windows, macOS і Linux, а також може бути викликаний через командний рядок, забезпечуючи гнучкість і універсальність.

Інструмент підтримує аналіз понад 30 мов програмування, таких як Java, Python, JavaScript, C++, C#, PHP, Ruby, Swift, Kotlin та інші. Він також може обробляти багатомовні проєкти, які містять код на декількох мовах.

SonarScanner легко інтегрується з популярними білд-системами, такими як Maven, Gradle та Ant, і може бути налаштований для автоматичного запуску під час створення білду. Крім того, він підтримує інтеграцію з CI/CD пайплайнами, такими як Jenkins, GitLab CI/CD, GitHub Actions, Azure DevOps, забезпечуючи автоматичний аналіз кожного коміту або пул-реквесту.

Інструмент відзначається гнучкістю у налаштуванні, дозволяючи кастомізувати профілі якості для адаптації аналізу до специфічних вимог проєкту. Він може використовуватися як для локального аналізу, так і в рамках автоматизованих процесів.

Принцип роботи SonarScanner

Процес роботи SonarScanner можна розбити на кілька етапів:

1) Збір вихідного коду: сканер збирає весь вихідний код із проєкту. Це може бути виконано локально з робочої станції розробника або з репозиторію, що зберігається у системах контролю версій, таких як Git або SVN;

2) Статичний аналіз: на основі налаштованого профілю якості (Quality Profile) виконується аналіз коду, включаючи:

- пошук багів, вразливостей, дублікацій та «поганих запахів» у коді;
- вимірювання метрик, таких як складність коду, покриття тестами, кількість рядків коду тощо;

3) Генерація звіту: після завершення аналізу SonarScanner створює звіт із результатами, який передається на сервер SonarQube.

4) Передача даних на сервер:

- сервер SonarQube отримує звіт і обробляє дані для візуалізації у веб-інтерфейсі;
- користувачі можуть переглядати результати аналізу, включаючи список проблем і рекомендації щодо їх усунення.

Варіанти використання SonarScanner

Запуск через командний рядок:

Прямий запуск із CLI дозволяє аналізувати код безпосередньо на робочій станції.

Приклад команди:

```
bash

sonar-scanner \
  -Dsonar.projectKey=my_project \
  -Dsonar.sources=src \
  -Dsonar.host.url=http://localhost:9000 \
  -Dsonar.login=your_token
```

Інтеграція з білд-системами:

Використовується у білд-скриптах Maven, Gradle або Ant.

Наприклад, для Maven це може бути:

```
mvn sonar:sonar -Dsonar.host.url=http://localhost:9000 -Dsonar.login=your_token
```

Інтеграція у CI/CD пайплайни:

Автоматичний аналіз кожного коміту через Jenkins, GitLab CI/CD або GitHub Actions.

Приклад конфігурації для GitLab CI/CD:

```
sonar_scanner:  
  stage: analyze  
  script:  
    - sonar-scanner -Dsonar.projectKey=my_project -Dsonar.source  
  only:  
    - main
```

SonarScanner забезпечує високу гнучкість, дозволяючи працювати з різними мовами програмування та середовищами. Інструмент відзначається швидкістю, виконуючи аналіз безпосередньо на робочій станції або в пайплайні, що мінімізує затримки. Його простота інтеграції дозволяє легко вбудувати SonarScanner в існуючі процеси розробки. Крім того, він підтримує автоматизацію, забезпечуючи автоматичний аналіз із кожним новим комітом або зміною у проєкті.

SonarScanner є критично важливим інструментом у роботі з SonarQube, що забезпечує ефективний аналіз коду та покращення його якості.

10.3. Quality Gates (Ворота якості)

Quality Gates – це один із найважливіших елементів у системі SonarQube, який забезпечує об'єктивну оцінку якості коду. Ворота якості встановлюють набір критеріїв, на основі яких ухвалюється рішення про те, чи відповідає проєкт стандартам якості. Вони дозволяють розробникам та командам автоматично контролювати якість коду на різних етапах розробки.

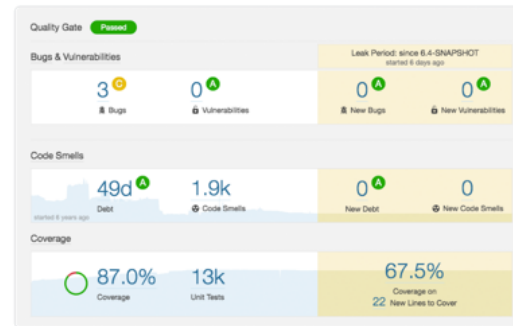
Що таке Quality Gates?

Quality Gates — набір критеріїв для автоматичної перевірки якості коду.

Приклади правил:

- Мінімальне тестове покриття: > 80%.
- Нуль критичних помилок і вразливостей.
- Відсутність нових багів.
- Якщо умови не виконуються, коміт/деплой може бути заблокований.

Мета: Забезпечити постійну відповідність високим стандартам якості.



Quality Gates спрямовані на забезпечення високої якості коду, виявляючи проблеми ще на ранніх етапах розробки, щоб мінімізувати їхній вплив на фінальний продукт. Вони дозволяють автоматизувати процес прийняття рішень, використовуючи метрики для оцінки успішності збірки чи випуску релізу. Постійний моніторинг ключових показників допомагає зменшувати технічний борг і підтримувати його на прийнятному рівні. Крім того, Quality Gates забезпечують відповідність коду корпоративним або галузевим стандартам якості.

Основні характеристики Quality Gates:

- метрики, що використовуються:

Quality Gates базуються на ключових показниках якості коду, серед яких:

- *покриття тестами*: відсоток коду, перевірений автоматизованими тестами;
- *кількість багів*: визначення проблем у коді, що можуть викликати некоректну роботу програми;
- *вразливості*: аналіз безпеки для виявлення можливих точок злому (наприклад, SQL-ін'єкції, XSS);
- *технічний борг*: час, необхідний для усунення проблем у коді;
- *дублікація коду*: виявлення повторюваних блоків коду, які можуть ускладнювати підтримку проекту;

- *складність коду*: оцінка когнітивної або цикломатичної складності коду для забезпечення його зрозумілості.
- *автоматизація*: Quality Gates інтегруються в CI/CD пайплайни, що дозволяє автоматично приймати рішення про успішність збірки або пул-реквесту. У разі невідповідності встановленим критеріям процес збірки може бути автоматично зупинений;
- *гнучкість*:
 - Quality Gates можуть бути налаштовані відповідно до потреб проекту або організації.
 - Дозволяють задавати різні профілі якості для різних типів проектів або команд.
- *результати в реальному часі*: усі результати перевірки воріт якості доступні у веб-інтерфейсі SonarQube, що дозволяє командам швидко реагувати на проблеми.

Приклади правил Quality Gates

Стандартний набір правил:

- покриття тестами має бути не менше 80%;
- кількість критичних багів та вразливостей має бути 0;
- дублікація коду повинна бути нижче 5%;
- технічний борг не повинен перевищувати 10% від загального часу розробки.

Правила для нового коду (New Code):

- покриття тестами нового коду має становити не менше 90%;
- жодних критичних або блокуючих проблем;
- вразливості безпеки на новому коді повинні бути відсутніми.

Правила для релізу:

- усі блокуючі баги мають бути усунені;
- загальна кількість технічного боргу не повинна перевищувати визначений поріг (наприклад, 20 годин);

- покриття тестами має бути стабільним або зростати.

Приклад використання Quality Gates у CI/CD пайплайнах

Jenkins

Під час виконання пайплайну Jenkins запускається SonarScanner для аналізу коду, після чого перевіряються результати Quality Gates:

```
stage('Quality Gate') {  
  steps {  
    script {  
      def qualityGate = waitForQualityGate()  
      if (qualityGate.status != 'OK') {  
        error "Pipeline failed due to Quality Gate failure:  
${qualityGate.status}"  
      }  
    }  
  }  
}
```

GitLab CI/CD

Quality Gates можуть бути інтегровані як окремий етап у пайплайні GitLab:

yaml

```
sonar_quality_gate:  
  stage: analyze  
  script:  
    - sonar-scanner -Dsonar.projectKey=my_project -Dsonar.sources=src  
  allow_failure: false
```

GitHub Actions

Використання Quality Gates через SonarCloud:

```
jobs:  
  sonarcloud:
```

```

runs-on: ubuntu-latest
steps:
  - name: Checkout code
    uses: actions/checkout@v3
  - name: SonarCloud Scan
    uses: sonarsource/sonarcloud-github-action@v2
  env:
    SONAR_TOKEN: ${ secrets.SONAR_TOKEN }
  - name: Quality Gate Check
    run: |
      if [ "$(curl -s -u $SONAR_TOKEN:
'https://sonarcloud.io/api/qualitygates/project_status?projectKey=my_project' | jq -r
'.projectStatus.status')" != "OK" ]; then
        exit 1
      fi

```

Quality Gates забезпечують об'єктивну оцінку якості коду, створюючи єдиний підхід для всієї команди. Вони дозволяють діагностувати критичні проблеми ще до релізу, що знижує ризик помилок у продакшні. Завдяки автоматизації Quality Gates ефективно інтегруються у DevOps-процеси, забезпечуючи безперервний моніторинг якості. Крім того, вони підтримують безпеку, контролюючи вразливості та підвищуючи захищеність програмного забезпечення.

Quality Gates – це важливий інструмент забезпечення якості коду, що сприяє стандартизації, автоматизації та підвищенню безпеки у процесах розробки. Завдяки своїй гнучкості вони можуть бути налаштовані відповідно до потреб кожної команди або проекту.

Приклади застосування SonarQube

SonarQube є універсальним інструментом, який активно використовується у різних сценаріях розробки, тестування та забезпечення якості коду. Ось кілька розширених прикладів його застосування:

Інтеграція з Jenkins

SonarQube легко інтегрується з Jenkins, дозволяючи автоматизувати аналіз якості коду на кожному етапі CI/CD пайплайну.

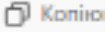
Сценарій роботи:

- після кожного коміту в репозиторій Jenkins запускає завдання для збірки проекту;
- під час білду викликається SonarScanner для аналізу коду;
- ворота якості (Quality Gates) перевіряють результати аналізу. Якщо код не відповідає критеріям, процес збірки блокується.

Переваги:

- автоматичне виявлення критичних помилок ще до релізу;
- забезпечення відповідності коду стандартам якості.

Приклад конфігурації:



```
pipeline {
  agent any
  stages {
    stage('Build') {
      steps {
        sh 'mvn clean install'
      }
    }
    stage('Code Analysis') {
      steps {
        withSonarQubeEnv('SonarQube') {
          sh 'mvn sonar:sonar'
        }
      }
    }
    stage('Quality Gate') {
      steps {
        script {
          def qualityGate = waitForQualityGate()
          if (qualityGate.status != 'OK') {
            error "Quality Gate failed: ${qualityGate.status}"
          }
        }
      }
    }
  }
}
```

CI/CD пайплайни

SonarQube інтегрується з різними системами автоматизації, такими як GitHub Actions, GitLab CI/CD, Bitbucket Pipelines, Azure DevOps, забезпечуючи безперервний аналіз коду.

GitHub Actions:

- під час створення пул-реквесту SonarQube виконує аналіз змін;
- якщо новий код порушує встановлені стандарти якості, об'єднання блокується.

GitLab CI/CD:

- SonarQube автоматично аналізує код перед злиттям змін в основну гілку (main branch).

Приклад використання GitHub Actions:

```
yaml

name: SonarQube Analysis
on:
  push:
    branches:
      - main
  pull_request:
    branches:
      - main
jobs:
  analyze:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout code
        uses: actions/checkout@v3
      - name: SonarQube Scanner
        uses: sonarsource/sonarcloud-github-action@v2
        env:
          SONAR_TOKEN: ${ secrets.SONAR_TOKEN }
```

Аналіз легасі-коду

SonarQube є потужним інструментом для роботи з існуючими (легасі) проектами, які накопичили значну кількість технічного боргу.

Сценарій роботи:

- виконується аналіз великої кодової бази, щоб отримати звіт про її стан;
- визначаються найпроблемніші модулі або файли, які потребують рефакторингу;

- пріоритетність виправлень встановлюється на основі технічного боргу та критичності проблем.

Переваги:

- допомагає командам сфокусуватися на найважливіших частинах коду;
- знижує ризик проблем, пов'язаних із підтримкою та масштабуванням.

Приклад:

Компанія може використовувати SonarQube для створення звіту, який покаже, що, наприклад, 20% файлів містять 80% проблем. Це дозволить команді зосередитися на цих 20% для максимального ефекту.

Кібербезпека та DevSecOps

SonarQube використовується як частина процесів DevSecOps для виявлення та усунення вразливостей у коді.

Сценарій роботи:

- виконується аналіз коду для виявлення потенційно небезпечних елементів, таких як SQL-ін'єкції, XSS (міжсайтовий скриптинг), неправильна обробка автентифікації тощо;
- вразливості класифікуються за рівнем критичності, і розробники отримують рекомендації щодо їх усунення.

Переваги:

- знижує ризик атак на програмне забезпечення;
- додає безпеку в процеси розробки без значного збільшення часу.

Приклад:

У банківському програмному забезпеченні SonarQube може бути використаний для пошуку SQL-ін'єкцій у модулях, пов'язаних із транзакціями.

Підтримка мобільної розробки

SonarQube також застосовується для аналізу якості коду мобільних додатків для платформ Android та iOS.

Сценарій роботи:

- аналіз Java/Kotlin коду для Android-додатків;

- аналіз Swift/Objective-C для iOS-додатків;
- виявлення проблем, які можуть вплинути на продуктивність або стабільність мобільного додатку.

Переваги:

- дозволяє виявити проблеми ще до того, як додаток потрапить у руки користувачів;
- підвищує якість додатків, що особливо важливо для конкурентного мобільного ринку.

SonarQube є незамінним інструментом у процесах забезпечення якості коду, кібербезпеки та автоматизації розробки. Його широкий спектр застосування робить його важливим елементом як для нових проектів, так і для роботи з легасі-кодом.